

PHASE

8

Processes, Context Switching and Scheduling

Run many things at once on one CPU: kernel threads, the context switch, and a preemptive scheduler driven by the timer

Overall progress toward Turk-OS 1.0 (this phase in red)

48 % → 56 %

🕒 Duration

3 weeks

★ Difficulty

★★★★★ (5/5)

✂ You need

Phase 7: `kmalloc`, `list.h`, `make test`; Phase 4: the timer interrupt

🔧 You will have

Kernel threads, each with its own stack and guard page, a context switch in about ten lines of assembly, cooperative `yield`, then preemptive round-robin scheduling on the timer, `sleep_ms` with proper blocking, an idle task that halts the CPU, task exit with reaping, and a `ps` command. Optionally, a multi-level feedback queue.

1 Why this phase

This is where Turk-OS stops being a program and becomes an operating system. A single CPU can only run one instruction stream at a time, but by switching between streams quickly enough, the kernel creates the illusion that many run at once. **OSTEP** calls this *virtualizing the CPU*, and it is the first of its three big ideas.

You will build it in two layers, as the books do. The **mechanism** is the context switch: saving everything that describes one running thread and restoring another. The **policy** is the scheduler: deciding which thread runs next and for how long. You start with kernel threads, which all share the kernel's address space; in Phase 10 the same machinery switches between user processes with separate address spaces. Take your time with the context switch. It is short, it is subtle, and every later phase depends on it.

2 Concepts you need

2.1 The process abstraction

OSTEP chapter 4 defines a process by its **machine state**: the registers (including the instruction and stack pointers), the memory it can address, and the I/O resources it holds. The kernel keeps this state in a **process control block** (PCB), which Turk-OS calls `struct task`, and tracks every task's **state** (Figure 1). A task is *running* on the CPU, *ready* to run and waiting its turn, *blocked* waiting for an event (a timer, a key, a disk block), or a *zombie* that has finished but whose resources have not been reclaimed yet.

2.2 The context switch

The key insight of **OSTEP** chapter 6 is that the kernel regains control of the CPU in only two ways: the running code calls into it (a system call, or `yield` for kernel threads), or a **timer interrupt** forces it in. Either way, the kernel is now running on the current task's kernel stack. To switch, it saves the current task's registers, records its stack pointer in its `struct task`, loads the next task's stack pointer, and restores that task's registers. Because each task has its **own kernel stack**, the part of the old task's state that lives on the stack (its return addresses, its interrupt frame) simply stays there until it is resumed.

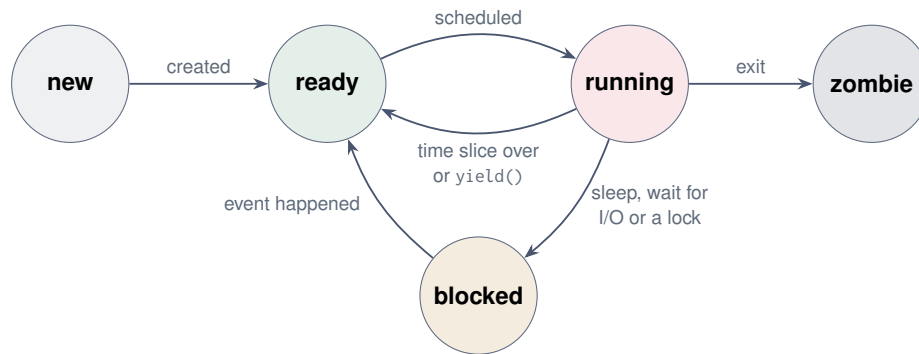


Figure 1: Task states in Turk-OS. Only the scheduler moves a task into *running*.

The switch is done by a C-callable assembly function, `switch_context`. It only needs to save the four callee-saved registers (EBX, ESI, EDI, EBP) plus ESP: under `cdecl`, the C code that calls it has already saved anything else it cares about (Phase 1). When the new task's stack is loaded and `switch_context` executes `ret`, it returns into wherever *that* task called `switch_context` from, which might be the middle of a timer interrupt from several seconds ago (Figure 2).

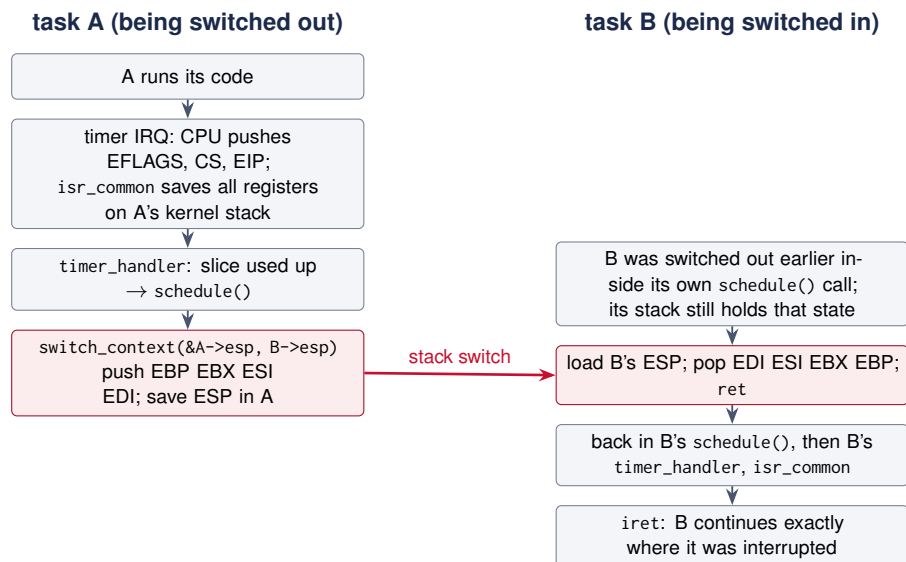


Figure 2: A preemptive switch from A to B. A's half-finished interrupt stays frozen on A's stack until A is chosen again.

2.3 Scheduling policy

With the mechanism in place, the scheduler decides who runs. **OSTEP** chapter 7 introduces the metrics: **turnaround time** (from arrival to completion) and **response time** (from arrival to first run). Policies like shortest-job-first optimise turnaround but need to know the future. **Round-robin** (RR) gives each ready task a fixed **time slice** (quantum) in turn, which is excellent for response time and is what Turk-OS starts with. Chapter 8's **multi-level feedback queue** (MLFQ) keeps several RR queues at different priorities, lowers the priority of tasks that use their whole slice (CPU-bound work), and periodically boosts everyone to prevent starvation. It favours interactive tasks without knowing anything about them in advance, which is why variants of it ran in most UNIX systems. **OSDI** §2.4.5 adds the principle to keep policy and mechanism separate, so you can change one without touching the other.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	OSTEP	Ch. 4 <i>The Abstraction: The Process</i>	25–36	Machine state, process states, the process list.
REQUIRED	OSTEP	Ch. 6 <i>Limited Direct Execution</i> , re-read §6.3 <i>Switching Between Processes</i> with its timeline	52–58	The exact mechanism of Tasks 8.3–8.6.
REQUIRED	OSTEP	Ch. 7 <i>Scheduling: Introduction</i> ; Ch. 8 <i>Scheduling: The Multi-Level Feedback Queue</i>	63–86	Metrics, RR, and the MLFQ rules for Task 8.9.
REQUIRED	LOB	Ch. 14 <i>Multitasking</i> : creating processes, cooperative and preemptive scheduling, separate kernel stacks	73–75	The x86 specifics, and the difficulties it warns about.
RECOMMENDED	MOS	§2.1 <i>Processes</i> (model, creation, states, implementation)	85–97	Another view, with the interrupt-handling sequence spelled out.
RECOMMENDED	MOS	§2.4 <i>Scheduling</i>	148–167	Goals per system type; RR, priority, multiple queues, lottery.
RECOMMENDED	OSDI	§2.5 <i>Overview of Processes in MINIX 3</i> ; §2.6.5 <i>Process Data Structures</i>	112–125, 146–156	MINIX’s process table and scheduling queues.
RECOMMENDED	OSDI	§2.6.10 <i>Scheduling in MINIX 3</i>	182–185	A real multi-queue scheduler in a few pages.
RECOMMENDED	OSC	§3.1–3.2 <i>Process Concept, Process Scheduling</i> ; §5.1–5.3 <i>CPU scheduling</i>	106–116, 200–217	Queues, context switches, algorithms with examples.
OPTIONAL	OSTEP	Ch. 9 <i>Proportional Share</i> ; Ch. 10 <i>Multiprocessor Scheduling</i>	87–108	Lottery and stride scheduling; what changes with more CPUs.
OPTIONAL	MOS	§2.2 <i>Threads</i> , especially §2.2.5 <i>Implementing Threads in the Kernel</i>	97–113	What you are building, placed among the alternatives.
OPTIONAL	OSC	§5.7 <i>Operating-System Examples</i> (Linux CFS, Windows)	234–244	How production schedulers do it.

Q Real-system lens: how MINIX 3 does it

MINIX keeps each process’s saved registers *in the process table* (`p_reg` in `kernel/proc.h` (05500)), not on a per-process kernel stack. Its assembly entry code in `kernel/mpx386.s` (06300) saves the interrupted process’s registers into its table entry, the kernel runs on one shared stack, and restart loads whichever process was chosen. `enqueue`, `dequeue` and `pick_proc` in `kernel/proc.c` (07400) manage 16 priority queues. This works for MINIX because its kernel never blocks in the middle of a call: work that has to wait is done by user-space servers. Turk-OS gives every task its own kernel stack, like Linux and xv6, so a task can block anywhere inside the kernel (in the middle of a disk read, for example) and resume there later.

4 Build it

Task 8.1 The task structure

Define struct `task` and a global list of all tasks (for `ps`), plus a current pointer. Keep the saved stack pointer as the first field so the assembly never needs to know the struct layout. Hand out PIDs from a counter.

```
include/task.h

enum task_state { TASK_READY, TASK_RUNNING, TASK_BLOCKED, TASK_ZOMBIE };

struct task {
    uint32_t    esp;           /* saved kernel ESP while not running (offset 0) */
    uint32_t    pid;
    enum task_state state;
    char        name[16];
    void        *kstack;       /* the stack allocation, freed when reaped */
    uint32_t    wake_tick;     /* when a sleeping task should wake */
    int         slice;         /* ticks left in the current time slice */
    uint32_t    cpu_ticks;     /* accounting for ps */
    struct list_node node;     /* on the run queue, a sleep queue, or zombies */
    struct list_node all;      /* on the list of all tasks */
};

extern struct task *current;
```

Task 8.2 Kernel stacks with guard pages

Give each task a 16 KiB kernel stack in the region from `0xF0000000`, whose page tables already exist. Divide the region into 20 KiB slots and map only the top four pages of each slot, leaving the lowest page unmapped. A stack overflow then hits the guard page and produces a page fault with an address just below the stack, instead of silently overwriting the next task's memory. Keep freed slots on a list for reuse.

✓ **Verify:** A test task that recurses without limit dies with a page fault whose address is in its own guard page.

Task 8.3 `switch_context`

Type this in and comment it until you could explain every line to someone else.

```
kernel/arch/i386/switch.s

; void switch_context(uint32_t *save_esp, uint32_t load_esp)
; Saves the current task's callee-saved registers and ESP, then resumes the task
; whose saved ESP is load_esp. Must be called with interrupts disabled.
global switch_context
switch_context:
    mov     eax, [esp + 4]      ; eax = &old->esp
    mov     edx, [esp + 8]     ; edx = new->esp
    push    ebp                ; save callee-saved registers on the OLD task's stack
    push    ebx
    push    esi
    push    edi
    mov     [eax], esp         ; old->esp = current stack pointer
    mov     esp, edx           ; switch to the NEW task's stack
    pop     edi                ; restore the new task's callee-saved registers
    pop     esi
    pop     ebx
    pop     ebp
    ret                        ; return to wherever the new task last called switch_context
```

Task 8.4 Creating a task

A new task has never called `switch_context`, so fake it: build its stack so the `pops` and `ret` land in a small trampoline. The trampoline enables interrupts (the new task doesn't return through an interrupt path that would restore them), calls the task's entry function, and calls `task_exit` if the function ever returns.

kernel/proc/task.c

```
extern void task_trampoline(void);           /* in switch.s, see below */

struct task *task_create(const char *name, void (*entry)(void *), void *arg)
{
    struct task *t = kcalloc(1, sizeof *t);
    t->kstack = kstack_alloc();              /* 16 KiB + guard page */
    uint32_t *sp = (uint32_t *)((char *)t->kstack + KSTACK_SIZE);

    *--sp = (uint32_t)task_trampoline;      /* switch_context's ret jumps here */
    *--sp = 0;                              /* ebp */
    *--sp = (uint32_t)entry;                 /* ebx: the function to run */
    *--sp = (uint32_t)arg;                   /* esi: its argument */
    *--sp = 0;                              /* edi */
    t->esp = (uint32_t)sp;

    t->pid = next_pid++;
    strncpy(t->name, name, sizeof t->name - 1);
    make_ready(t);                          /* state = READY, append to run queue */
    return t;
}
```

kernel/arch/i386/switch.s (continued)

```
extern task_exit
global task_trampoline
task_trampoline:
    sti                ; new tasks start with interrupts enabled
    push esi           ; argument
    call ebx           ; entry(arg)
    call task_exit      ; if entry returns, end the task (never returns)
```

✓ **Verify:** In GDB, break on `task_trampoline`: when it is hit, `EBX` holds your entry function and `ESI` its argument.

Task 8.5 Cooperative multitasking

Turn the code that is already running (`kmain`) into task 0 by giving it a `struct task`; its `esp` gets filled in at the first `switch`. Write `schedule()`, which moves the current task to the back of the ready queue if it is still runnable and switches to the task at the front, and `yield()`, which calls it with interrupts disabled. Save and restore the flags rather than blindly using `sti`, because `yield` may be called with interrupts already off.

kernel/proc/sched.c

```
static struct list_node run_queue;          /* READY tasks, first in first out */

void schedule(void)                        /* interrupts must be disabled */
{
    struct task *prev = current;
    if (prev->state == TASK_RUNNING && prev != idle_task) {
        prev->state = TASK_READY;
        list_add_tail(&run_queue, &prev->node);
    }
}
```

```

    struct task *next = idle_task;
    if (!list_empty(&run_queue)) {
        next = container_of(run_queue.next, struct task, node);
        list_del(&next->node);
    }
    next->state = TASK_RUNNING;
    next->slice = QUANTUM_TICKS;          /* e.g. 5 ticks = 50 ms at 100 Hz */
    if (next == prev)
        return;
    current = next;
    switch_context(&prev->esp, next->esp);
}

void yield(void)
{
    uint32_t flags;
    __asm__ volatile ("pushf; pop %0; cli" : "=r"(flags));
    schedule();
    if (flags & 0x200)                /* IF was set before: enable again */
        __asm__ volatile ("sti");
}

```

✔ **Verify:** Two tasks that each print a letter and call `yield()` in a loop produce `ABABAB...`; a third task joins the rotation.

Task 8.6 Preemption from the timer

Now let the timer take the CPU away. In the timer handler, charge the tick to current, and when its slice reaches zero, call `schedule()`. One change elsewhere is essential: in `isr_handler`, send the EOI to the PIC *before* calling an IRQ handler, not after. If the handler switches to another task, the old task's half-finished interrupt stays frozen on its stack, and an EOI placed after the call would not be sent until that task runs again. Until then the PIC delivers no more timer interrupts and preemption stops. Sending it first is safe because the interrupt gate keeps interrupts disabled until `iret`.

✔ **Verify:** Three tasks that loop printing without ever calling `yield` still take turns. The kernel monitor stays responsive while they run.

Task 8.7 Sleeping, idling and exiting

Add `sleep_ms`: set `wake_tick`, mark the task blocked, put it on a sleep list, and call `schedule()`. The timer handler moves every task whose `wake_tick` has passed back to the ready queue. Create an idle task that runs `sti; hlt forever`; `schedule` picks it only when nothing else is ready, so the CPU halts instead of spinning. Finally, `task_exit` marks the task as a zombie, puts it on a zombie list and schedules away. It can't free its own stack, because it is still running on it, so another task (the idle task is a good choice) reaps zombies: it frees their stacks and structures.

✔ **Verify:** With every task sleeping, QEMU's CPU usage on the host drops to almost zero. Creating and exiting 1000 tasks leaves the PMM and heap statistics exactly where they started.

Task 8.8 ps and accounting

Add a `ps` monitor command that walks the list of all tasks and prints PID, name, state and CPU ticks. Add `spawn <name>` to start a demo task and `kill <pid>` to end one (for kernel threads, simply mark it to exit at its next tick).

Task 8.9 A multi-level feedback queue (optional)

Replace the single run queue with three queues and apply the rules of **OSTEP** chapter 8: the highest non-empty queue runs first; equal priorities share round-robin; new tasks start at the top; a task that

uses up its allotment at a level moves down; every second, all tasks move back to the top. Compare a CPU-bound task and an interactive one (the monitor) with and without MLFQ.

5 Testing and debugging

When a switch goes wrong, the symptom usually appears later, in the new task. Break on `switch_context`, step through it with `stepi`, and check with `x/6xw $esp` that the new stack holds exactly what the `pops` and `ret` expect. The GDB command `info symbol <addr>` tells you where a return address points.

Symptom	Likely cause	What to do
Preemption works once, then the timer stops	EOI sent after the IRQ handler, which switched away	Send the EOI before calling the handler (Task 8.6).
New task crashes at its first instruction	The initial stack doesn't match <code>switch_context</code> 's pop order	Five words: EDI, ESI, EBX, EBP, return address, from low to high.
New task runs, but never gets preempted	Interrupts disabled in the new task	The trampoline must execute <code>sti</code> .
Random corruption, other tasks' data changes	Kernel stack overflow into a neighbour	Guard pages turn this into a page fault; make stacks bigger or recursion shallower.
Crash in <code>task_exit</code> or right after	The task freed its own stack while running on it	Only mark it as a zombie; let another task reap it.
Queue corruption under load	<code>schedule()</code> entered with interrupts enabled, so a timer interrupt re-entered it	Every path into <code>schedule()</code> disables interrupts first; assert IF is clear at its start.

6 Milestone

Milestone: Turk-OS multitasks

- ☐ Kernel threads run with their own 16 KiB stacks and guard pages.
- ☐ `switch_context` and `task_trampoline` work; you can explain every instruction.
- ☐ At least three threads share the CPU preemptively, without calling `yield`.
- ☐ `sleep_ms` blocks properly; the idle task halts the CPU when nothing is ready.
- ☐ Tasks exit and are reaped; 1000 create/exit cycles leak no memory.
- ☐ `ps` shows every task with its state and CPU time.

Self-check questions

1. What state must be saved in a context switch, and why does `switch_context` save only four registers besides ESP?
2. How does the timer interrupt make preemption possible, according to **OSTEP** chapter 6?
3. Draw the stack of a newly created task just before its first switch.
4. Why must the EOI be sent before an IRQ handler that might call `schedule()`?
5. Round-robin versus MLFQ: what problem does MLFQ solve, and why does it need a periodic priority boost?
6. Why can't a task free its own kernel stack in `task_exit`?
7. MINIX saves registers in the process table and uses a single kernel stack. What can Turk-OS do that MINIX's kernel can't, and what does it cost?

7 Going further

Implement lottery or stride scheduling from **OSTEP** chapter 9 and compare fairness over a long run. Add a top command that shows CPU usage per task over the last second. Measure the cost of a context switch by switching back and forth a million times and reading the CPU's time-stamp counter (`rdtsc`) before and after. If you are heading for the SMP track in Phase 15, read **OSTEP** chapter 10 now and note what would have to change: one current per CPU, per-CPU run queues, and real locks.

Next: **Phase 9 Synchronization and Concurrency in the Kernel** →

Books used in this plan

- LOB** Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at <https://littleosbook.github.io>.
- OSTEP** Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at <https://pages.cs.wisc.edu/~remzi/OSTEP/>.
- OSC** A. Silberschatz, P.B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.
- MOS** Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.
- OSDI** Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

The books do not come with this plan. **LOB** and **OSTEP** are free online; buy or borrow the other three (the roadmap lists their ISBNs). Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.